Requirements Analysis and Specification (Lecture 3A)

Prof (Dr.) H S Behera

Organization of this Lecture

- Brief review of previous lectures
- Introduction
- Requirements analysis
- Requirements specification
- SRS document
- Decision table
- Decision tree
- Summary

Requirements Analysis and Specification

- Many Projects Fail:
  - Because they Start Implementing the System:
    - ✓ without determining whether they are building what the customer wants

Requirements Analysis and Specification

- It is important to learn:
  - requirements analysis and specification techniques thoroughly.

Requirements Analysis and Specification

- Requirement analysis and specification is a critical software engineering phase that translates user needs into a structured, actionable Software Requirements Specification (SRS) document.
- It involves gathering requirements, analyzing them for consistency and completeness, and documenting them to guide development.
- Key activities include elicitation, analysis and formalizing functional/non-functional requirements.

Requirements Analysis and Specification

- Requirement analysis and specification is the systematic process of gathering, analyzing, and documenting user needs, resulting in a **Software Requirements Specification (SRS) document**.
- It transforms high-level goals into detailed functional and non-functional requirements, ensuring accuracy, consistency, and verifiability to prevent scope creep.

Requirements Analysis and Specification

- Goals of requirements analysis and specification phase:
  - fully understand the user requirements
  - remove inconsistencies, anomalies, etc. from requirements
  - document the requirements properly in an SRS document

Requirements Analysis and Specification

- **Consists of two distinct activities:**
  - ✓ **Requirements Gathering and Analysis**
  - ✓ **Specification**

Requirements Analysis and Specification

- The person who undertakes requirements analysis and specification:
  - known as Systems Analyst:
  - collects data pertaining to the product
  - analyses collected data:
    - to understand what exactly needs to be done.
  - Writes the Software Requirements Specification (SRS) document.

Requirements Analysis and Specification

- Final output of this phase:
  - **Software Requirements Specification (SRS) Document**.
- The SRS document is reviewed by the customer.
  - **reviewed SRS document forms the basis of all future development activities.**

Requirements Analysis

- Requirements analysis consists of two main activities:
  - Requirements gathering
  - Analysis of the gathered requirements

Requirements Analysis

- Analyst gathers requirements through:
  - observation of existing systems,
  - studying existing procedures,
  - discussion with the customer and end-users,
  - analysis of what needs to be done, etc.

Requirements Gathering

- **If the project is to automate some existing procedures**
  - e.g., automating existing manual accounting activities,
  - the task of the system analyst is a little easier
  - analyst can immediately obtain:
    - input and output formats
    - accurate details of the operational procedures

Requirements Gathering (CONT.)

- In the absence of a working system,
  - ✓lot of imagination and creativity are required.
- Interacting with the customer to gather relevant data:
  - ✓requires a lot of experience.

Requirements Gathering (CONT.)

- Some desirable attributes of a good system analyst:
  - **Good interaction skills,**
  - **Imagination and Creativity,**
  - **Experience.**

Analysis of the Gathered Requirements

- After gathering all the requirements:
  - **analyze it:**
    - ✓ **Clearly understand the user requirements,**
    - ✓ **Detect inconsistencies, ambiguities, and incompleteness.**
- Incompleteness and inconsistencies:
  - **resolved through further discussions with the end-users and the customers**.

Inconsistent requirement

- **Some part of the requirement:**
  - **contradicts with some other part.**
  - Example:
    - **One customer says turn off heater and open water shower when temperature > 100 C**
    - **Another customer says turn off heater and turn ON cooler when temperature > 100 C**

Incomplete requirement

- Some requirements have been omitted:
  - ✓due to oversight.
- Example:
  - ➤The analyst has not recorded: when temperature falls below 90 C
    - ✓heater should be turned ON
    - ✓water shower turned OFF.

Analysis of the Gathered Requirements (CONT.)

- **Requirements analysis involves:**
  - obtaining a clear, in-depth understanding of the product to be developed,
  - remove all ambiguities and inconsistencies from the initial customer perception of the problem.

Analysis of the Gathered Requirements (CONT.)

- It is quite difficult to obtain:
  - **a clear, in-depth understanding of the problem:**
    - ✓especially if there is no working model of the problem.

Analysis of the Gathered Requirements (CONT.)

- **Experienced Analysts take considerable time:**
  - **to understand the exact requirements the customer has in his mind.**

Analysis of the Gathered Requirements (CONT.)

- **Experienced systems analysts know**
  - often as a result of painful experiences
    - without a clear understanding of the problem, it is impossible to develop a satisfactory system.

Analysis of the Gathered Requirements(CONT.)

- Several things about the project should be clearly understood by the analyst:
  - What is the problem?
  - Why is it important to solve the problem?
  - What are the possible solutions to the problem?
  - What complexities might arise while solving the problem?

Analysis of the Gathered Requirements(CONT.)

- **Some anomalies and inconsistencies can be very subtle/understated:**
  - escape even most experienced eyes.
  - If a formal model of the system is constructed,
    - ✓many of the subtle anomalies and inconsistencies get detected.

Analysis of the Gathered Requirements(CONT.)

- After collecting all data regarding the system to be developed,
  - remove all inconsistencies and anomalies from the requirements,
  - systematically organize requirements into a Software Requirements Specification (SRS) document.

Software Requirements Specification

- **Main aim of requirements specification:**
  - Systematically organize the requirements arrived during requirements analysis
  - Document requirements properly.

Software Requirements Specification

- The SRS document is useful in various contexts:
  - Statement of user needs
  - Contract document
  - Reference document
  - Definition for implementation

Software Requirements Specification: A Contract Document

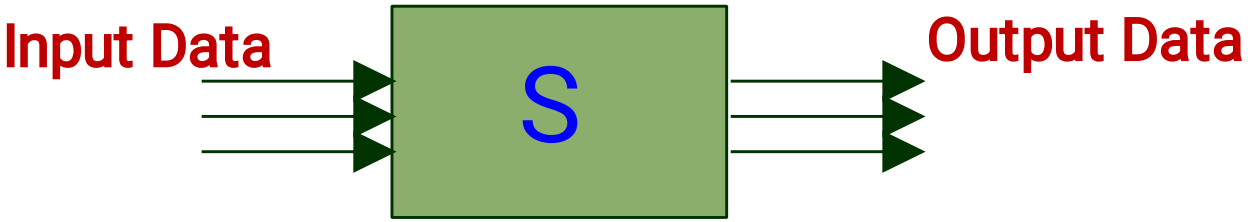
- **Requirements document is a reference document.**
- SRS document is a contract between the development team and the customer.
  - **Once the SRS document is approved by the customer,**
    - **any subsequent controversies are settled by referring the SRS document.**

Software Requirements Specification: A Contract Document

- Once customer agrees to the SRS document:
  - development team starts to develop the product according to the requirements recorded in the SRS document.
- The final product will be acceptable to the customer:
  - as long as it satisfies all the requirements recorded in the SRS document.

SRS Document (CONT.)

- The SRS document is known as **black-box specification:**
  - the system is considered as a black box whose internal details are not known.
  - only its visible external (i.e. input/output) behavior is documented.



SRS Document (CONT.)

- The SRS document is known as a black-box specification because it focuses exclusively on the system's externally visible behavior, functionality, inputs, and outputs, rather than its internal design or implementation details. It describes what the system does, not how it does it.
- An SRS document is known as a "black box specification" because it defines what a system must do (external behavior, inputs, and outputs) without detailing how to implement it (internal logic, algorithms, or code).
- It treats the system as a "black box" where only the inputs and resulting outputs are visible, masking internal complexity.

SRS Document (CONT.)

- SRS document concentrates on:
  - what needs to be done
  - carefully avoids the solution ("how to do") aspects.
- The SRS document serves as a contract
  - between development team and the customer.
  - Should be carefully written

SRS Document (CONT.)

- The requirements at this stage:
  - ✓written using end-user terminology.
- If necessary:
  - ✓later a formal requirement specification may be developed from it.

Properties of a good SRS document

- It should be concise
  - and at the same time should not be ambiguous.
- It should specify what the system must do
  - and not say how to do it.
- Easy to change.,
  - i.e. it should be well-structured.
- It should be consistent.
- It should be complete.

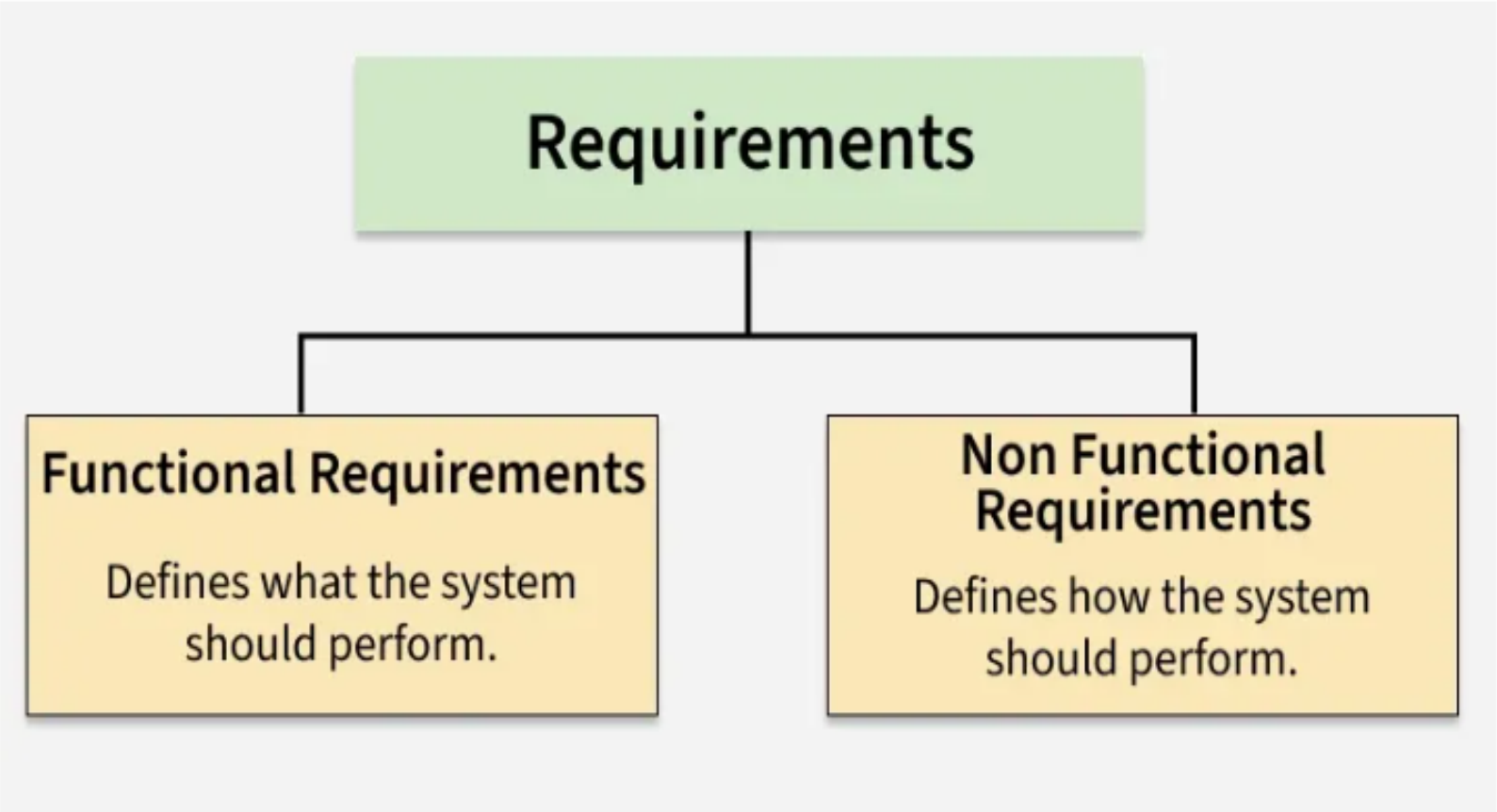
Properties of a good SRS document (cont...)

- It should be traceable
  - you should be able to trace which part of the specification corresponds to which part of the design and code, etc and vice versa.
- It should be verifiable
  - e.g. “the system should be user-friendly” are often not verifiable unless "user-friendly" is defined using quantifiable metrics (e.g., average time to complete a task, satisfaction score from a usability test).

SRS Document (CONT.)

- SRS document, normally contains three important parts:
  - functional requirements,
  - nonfunctional requirements,
  - constraints on the system.

Requirements



SRS Document (CONT.)

- **Functional Requirements:** These describe the specific, high-level functions that the system must perform, including how it transforms input data into output data. Functional requirements (FRs) define what a system must do (features, behavior, and user interactions)
- **Non-functional Requirements:** These define the system's quality attributes and behavioral characteristics that cannot be expressed as functions, such as performance (speed), security, reliability, usability, portability and maintainability. Non-functional requirements (NFRs) define how the system performs (quality, constraints, and constraints like speed, security, and scalability) .
- FRs are mandatory for operation, whereas NFRs ensure a high-quality, efficient user experience.

SRS Document (CONT.)

- **Constraints on the System:** These describe the limitations or restrictions under which the system must operate, such as hardware limitations, operating system, required standards compliance, and data representation restrictions.
- Constraints are rigid restrictions that limit the options of developers and architects. They are often non-negotiable, legal, or technical limitations.
- Examples:
  - Technical: "The application must be developed for iOS 15 or higher."
  - Budgetary: "The total development cost must not exceed ."
  - Regulatory: "The system must comply with GDPR(General Data Protection Regulation) data privacy regulations."
  - Time: "The project must be completed within 6/12 months."

Core Differences Summary

Core Differences Summary

Feature	Functional Requirements	Non-functional Requirements
Question	What does the system do?	How does the system do it?
Requirement Type	Mandatory features/actions	Quality attributes/constraints
Testing Type	Unit, Integration, System testing	Load, Security, Usability testing
Measurability	Discrete (Works / Does not work)	Measured by metrics (Speed, Uptime)
Architecture	Influences system design	Influences system architecture

Comparison Summary

Comparison Summary

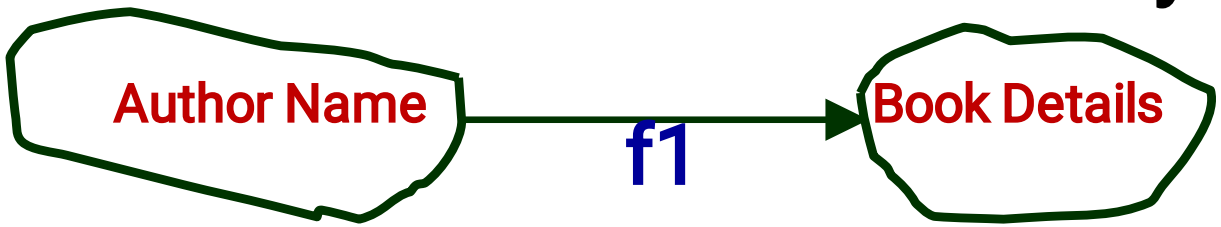
Feature	Functional Requirements	Non-Functional Requirements	Constraints
Defines	What the system does	How the system performs	Limits on development
Focus	System behavior/features	Quality attributes (e.g., speed)	Boundaries/restrictions
Examples	User Login, Print Report	3-second load time, Security	Budget, Legal, Technology
Tested By	User Acceptance Testing	Performance/Stress Testing	Compliance Check

SRS Document (CONT.)

- It is desirable to consider every system:
  - performing a set of functions {fi}.
  - Each function fi considered as:
  - transforming a set of input data to corresponding output data.

Example: Functional Requirement

- F1: Search Book
  - Input:
    - an author's name:
  - Output:
    - details of the author's books and the locations of these books in the library.



Functional Requirements

- Functional requirements (FRs) define the specific, mandatory behaviors, features, and functions a system must perform to meet user needs.

Functional Requirements

- **Functional requirements describe:**
  - A set of high-level requirements
  - Each high-level requirement:
    - takes in some data from the user
    - outputs some data to the user
  - Each high-level requirement:
    - might consist of a set of identifiable functions

Functional Requirements

- For each high-level requirement:
  - Every function is described in terms of
    - input data set
    - output data set
    - processing required to obtain the output data set from the input data set

Non-functional requirements

- Non-functional requirements (NFRs) define how a system operates, focusing on quality attributes like performance, security, usability, and reliability, rather than specific behaviors (what it does).

Nonfunctional Requirements

- Characteristics of the system which can not be expressed as functions:
  - maintainability,
  - portability,
  - usability, etc.

Nonfunctional Requirements

- Nonfunctional requirements include:
  - reliability issues,
  - performance issues,
  - human-computer interface issues,
  - Interface with other external systems,
  - security, maintainability, etc.

Constraints

- Constraints in (SRS) are binding limitations, restrictions, or boundaries that dictate how a system must be designed and built. They define " how" a system operates rather than "what" it does, encompassing technical, regulatory, budgetary, and time limitations.
- Key categories include, but are not limited to, hardware/software constraints, compliance standards, and performance thresholds

Constraints

- Constraints describe things that the system should or should not do.
  - For example,
    - standards compliance
    - how fast the system can produce results
      - so that it does not overload another system to which it supplies data, etc.

Examples of Constraints

- Hardware to be used,
- Operating system
  - ✓or DBMS to be used
- Capabilities of I/O devices
- Standards compliance
- Data representations
  - ✓by the interfaced system

Organization of the SRS Document

- Introduction.
- Functional Requirements
- Nonfunctional Requirements
  - External interface requirements
  - Performance requirements
- Constraints

Example Functional Requirements

- List all functional requirements
  - with proper numbering.
- Req. 1:
  - ✓Once the user selects the "search" option,
    - he is asked to enter the key words.
  - ✓The system should output details of all books
    - whose title or author name matches any of the key words entered.
    - Details include: Title, Author Name, Publisher name, Year of Publication, ISBN Number, Cata log Number, Location in the Library.

Example of Functional Requirements

- Req. 2:
  - ✓When the “renew” option is selected,
    - the user is asked to enter his membership number and password.
  - ✓After password validation,
    - the list of the books borrowed by him are displayed.
  - ✓The user can renew any of the books:
    - by clicking in the corresponding renew box.

Req. 1:

- R.1.1:
  - ✓ Input: "search" option,
  - ✓ Output: user prompted to enter the key words.
- R1.2:
  - ✓ Input: key words
  - ✓ Output: Details of all books whose title or author name matches any of the key words.
    - Details include: Title, Author Name, Publisher name, Year of Publication, ISBN Number, Cata log Number, Location in the Library.
  - ✓ Processing: Search the book list for the keywords

Req. 2:

- R2.1:
  - ✓Input: “renew” option selected,
  - ✓Output: user prompted to enter his membership number and password.
- R2.2:
  - ✓Input: membership number and password
  - ✓Output:
    - list of the books borrowed by user are displayed. User prompted to enter books to be renewed or
    - user informed about bad password
  - ✓Processing: Password validation, search books issued to the user from borrower list and display.

Req. 2:

- R2.3:
  - ✓Input: user choice for renewal of the books issued to him through mouse clicks in the corresponding renew box.
  - ✓Output: Confirmation of the books renewed
  - ✓Processing: Renew the books selected by the in the borrower list.

Examples of Bad SRS Documents

- Unstructured Specifications:
  - Narrative essay --- one of the worst types of specification document:
    - Difficult to change,
    - Difficult to be precise,
    - Difficult to be unambiguous,
    - Scope for contradictions, etc.

Examples of Bad SRS Documents

- **Noise:**
  - ✓Presence of text containing information irrelevant to the problem.
- **Silence:**
  - ✓aspects important to proper solution of the problem are omitted.

Examples of Bad SRS Documents

- **Overspecification:**
  - ✓Addressing "how to" aspects
  - ✓For example, "Library member names should be stored in a sorted descending order"
  - ✓Overspecification restricts the solution space for the designer.
- **Contradictions:**
  - ✓Contradictions might arise
    - if the same thing described at several places in different ways.

Examples of Bad SRS Documents

- **Ambiguity:**
  - ✓Literary expressions
  - ✓Unquantifiable aspects, e.g. “good user interface”
- **Forward References:**
  - ✓References to aspects of problem
    - defined only later on in the text.
- **Wishful Thinking:**
  - ✓Descriptions of aspects
    - for which realistic solutions will be hard to find.

Representation of complex processing logic

- Complex processing logic is represented using formal, structured methods like decision tables, decision trees, and, logic-based, geometric, or probabilistic embeddings.
- These methods translate intricate, often human-defined rules (declarative or procedural) into machine-interpretable formats—such as first-order logic and predicate calculus—to enable automation, reasoning, and consistent decision-making.
- Complex processing logic is represented using tools like decision tables, decision trees, and structured English to model intricate business rules, conditions, and actions.

Key Techniques for Representing Complex Logic

- **Decision Tables/Trees**: Ideal for mapping multiple conditions to specific actions, enhancing clarity for developers and testers.
- **Formal Logic**: Uses propositional logic, predicate calculus, and first-order logic (constants, variables, predicates, functions, quantifiers) to structure rules.
- **Computational Graphs**: Represents queries as nodes and edges, using geometric approaches or probability distributions.
- **Knowledge Representation (KR) Frameworks**: Employs declarative (facts), procedural (steps), heuristic (experience), or meta-knowledge to structure information.

Representation of complex processing logic:

- Decision trees
- Decision tables

Decision trees

- **Decision trees** are flowchart-like, hierarchical models used to represent complex, conditional logic, classification, or regression tasks by breaking down decisions into a series of binary (yes/no) or multi-way questions. They consist of a root node, internal decision nodes, branches, and leaf nodes (final outcomes).
- **Decision tree** is a graphical representation of processing logic that maps complex decision-making into a structured, hierarchical flow. It is particularly effective for modeling hierarchical decisions where outcomes depend on a series of interrelated conditions.

Key Components & Structure

- **Root Node**: The starting point representing the top-most question or main dataset.
- **Internal Nodes** (Decision Nodes): Points where a specific attribute is tested, branching based on the outcome.
- **Branches**: Connect nodes and represent the results of the decision.
- **Leaf Nodes** (Terminal Nodes): Represent the final decision, classification, or prediction

Decision Trees

- Decision trees:
  - ✓edges of a decision tree represent conditions
  - ✓leaf nodes represent actions to be performed.
- A decision tree gives a graphic view of:
  - ✓logic involved in decision making
  - ✓corresponding actions taken.

Example: LMS

- A Library Membership automation Software (LMS) should support the following three options:
  - ✓new member,
  - ✓renewal,
  - ✓cancel membership.

Example: LMS

- When the <u>new member</u> option is selected,
  - the software asks details about the member:
    - ✓name,
    - ✓address,
    - ✓phone number, etc.

Example(cont.)

- **If proper information is entered,**
  - ✓a membership record for the member is created
  - ✓a bill is printed for the annual membership charge plus the security deposit payable.

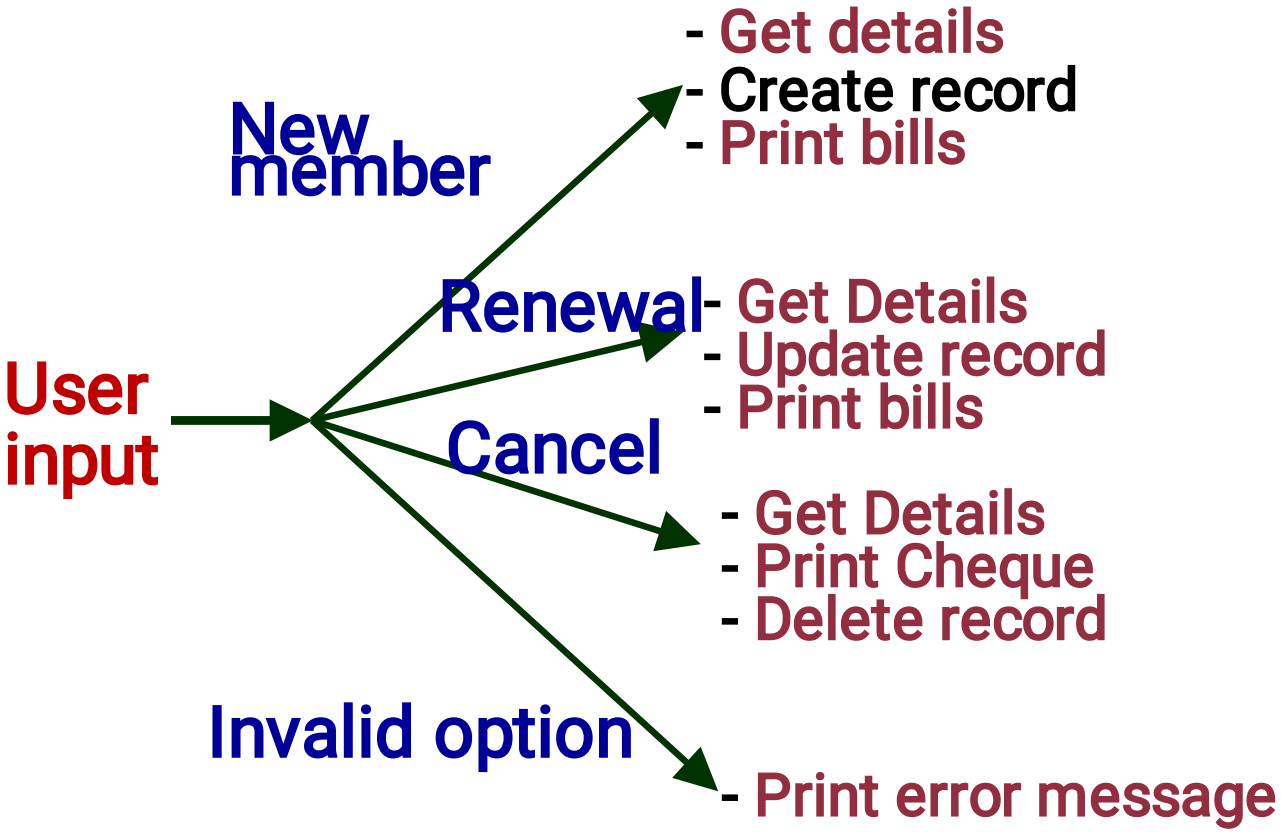
Example(cont.)

- If the renewal option is chosen,
  - LMS asks the member's name and his membership number
    - checks whether he is a valid member.
  - If the name represents a valid member,
    - the membership expiry date is updated and the annual membership bill is printed,
    - otherwise, an error message is displayed.

Example(cont.)

- If the cancel membership option is selected and the name of a valid member is entered,
  - ✔the membership is cancelled,
  - ✔a cheque for the balance amount due to the member is printed
  - ✔the membership record is deleted.

Decision Tree



Decision table

- **A decision table is a structured, matrix-based software engineering tool used to model complex logic, mapping multiple input conditions to specific actions or outputs.**
- It serves as a comprehensive technique in requirement analysis and black-box testing, ensuring all combinations of conditions are covered and reducing the risk of overlooked edge cases.

Components of a Decision Table

A decision table is divided into four main parts:

- **Condition Stubs:** List all input conditions (e.g., "User is logged in", "Item is in stock").
- **Action Stubs:** List all possible actions/outputs (e.g., " Proceed to checkout", "Show error message").
- **Condition Entries:** Specify combinations of conditions using True (T)/False (F) or Yes (Y)/No (N).
- **Action Entries:** Indicate which actions are performed for each combination.

Structure Example

Structure Example

Conditions/Actions	Rule 1	Rule 2	Rule 3
Condition 1	T	T	F
Condition 2	T	F	F
Action 1	X		
Action 2		X	X

- **Rules:** Each column represents a unique combination of conditions that leads to a specific set of actions (indicated by 'X') .
- **"Don't Care":** A dash (-) or blank can be used to indicate a condition that does not affect the output, reducing the number of rules needed

Decision Table

- **Decision tables specify:**
  - Which variables are to be tested
  - What actions are to be taken if the conditions are true,
  - The order in which decision making is performed.

Decision Table

- A decision table shows in a tabular form:
  - ✓processing logic and corresponding actions
- Upper rows of the table specify:
  - ✓the variables or conditions to be evaluated
- Lower rows specify:
  - ✓the actions to be taken when the corresponding conditions are satisfied.

Decision Table

- **In technical terminology,**
  - a column of the table is called a **rule**:
  - **A rule implies:**
    - ✓if a condition is true, then execute the corresponding action.

Example:

- Conditions

Valid selection	NO	YES	YES	YES
New member	--	YES	NO	NO
Renewal	--	NO	YES	NO
Cancellation	--	NO	NO	YES

- Actions

Display error message	--	--	--	
Ask member's name etc.				
Build customer record	--	--	--	
Generate bill	--	--		
Ask membership details	--			
Update expiry date	--	--	--	
Print cheque	--	--	--	
Delete record	--	--	--	

Structured English

- Structured English in software engineering is a, precise, plain-English, hybrid, pseudo-code used to define process logic without strict syntax. It uses keywords (IF, THEN, ELSE, DO, WHILE), indentation, and imperative sentences, making it ideal for defining complex algorithms or business rules.

1. The Problem in Structured English

A company wants to determine the discount percentage for a customer order based on the following rules:

- IF the customer has prepaid AND the order is over $1000, THEN give a 10% discount.
- IF the customer has prepaid AND the order is $1000 or less, THEN give a 5% discount.
- IF the customer has not prepaid AND the order is over $1000, THEN give a 2% discount.
- ELSE, give no discount.

2. Extract Conditions and Actions

- **Conditions (Inputs):**
  1. Prepaid? (Yes/No)
  2. Order > $1000? (Yes/No)
- **Actions (Outputs):**
  1. Discount 10%
  2. Discount 5%
  3. Discount 2%
  4. No Discount

Construct the Decision Table

We create a table with four rules to cover all combinations of the two conditions (columns).

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Prepaid?	Y	Y	N	N
Order > $1000?	Y	N	Y	N
Actions				
Give 10% Discount	X	-	-	-
Give 5% Discount	-	X	-	-
Give 2% Discount	-	-	X	-
No Discount	-	-	-	X

- **Y/N** = Yes/No
- **X** = Action to be taken
- **-** = Action not taken

4. Interpretation of Results

- Rule 1 (Y, Y): Prepaid is Yes, Order > 1000 is Yes -> 10% Discount.
- Rule 2 (Y, N): Prepaid is Yes, Order > 1000 is No -> 5% Discount.
- Rule 3 (N, Y): Prepaid is No, Order > 1000 is Yes -> 2% Discount.
- Rule 4 (N, N): Prepaid is No, Order > 1000 is No -> No Discount.

This table removes the complexity of nested "IF" statements, making it easier to verify that every scenario has an assigned outcome

Comparison

- **Both decision tables and decision trees**
  - ✓can represent complex program logic.
- **Decision trees are easier to read and understand**
  - ✓when the number of conditions are small.
- **Decision tables help to look at every possible combination of conditions.**

Formal specification techniques

- **Formal specification techniques** in software engineering use precise mathematical notations (logic, sets, functions) to define, analyze, and verify system behavior, eliminating ambiguity in requirements.
- These rigorous, model-based or algebraic methods enhance quality in critical systems, enabling early error detection and automated verification.
- Formal specification technique uses mathematically based notations to precisely describe the intended behavior, properties, and requirements of a system.
- Unlike informal natural language descriptions, formal specifications aim to eliminate ambiguity and allow for automated analysis and verification of system properties.

Formal Specification

- A formal specification technique is a mathematical method to:
  - accurately specify a system
  - verify that implementation satisfies specification
  - prove properties of the specification

Formal Specification

- Advantages:
  - ✔Well-defined semantics, no scope for ambiguity
  - ✔Automated tools can check properties of specifications
  - ✔Executable specification

Formal Specification

- Disadvantages of formal specification techniques:
  - ✓Difficult to learn and use
  - ✓Not able to handle complex systems

Formal Specification

- Mathematical techniques used include:
  - ✓Logic-based
  - ✓set theoretic
  - ✓algebraic specification
  - ✓finite state machines, etc.

Semiformal Specification

- Structured specification languages
  - SADT (Structured Analysis and Design Technique)
  - PSL/PSA (Problem Statement Language/ Problem Statement Analyzer)
    - ✔PSL is a semi-formal specification language
    - ✔PSA can analyze the specifications expressed in PSL

Executable Specification Language

- If specification is expressed in formal language:
  - ✔ it becomes possible to execute the specification to provide a system prototype.
- However, executable specifications are usually slow and inefficient.

Executable Specification Language

- Executable specifications only test functional requirements:
  - If non-functional requirements are important for some product,
    - ✔the utility of an executable specification prototype is limited.

4GLs

- 4GLs (Fourth Generation Languages) are examples of
  - ✓executable specification languages.
- 4GLs are successful
  - ✓because there is a lot of commonality across data processing applications.

4GLs

- 4GLs rely on software reuse
  - ✔where common abstractions have been identified and parameterized.
- Rewriting 4GL programs in higher level languages:
  - ✔result in up to 50% lower memory requirements
  - ✔also, the programs run up to 10 times faster.

Summary

- Requirements analysis and specification
  - ✓an important phase of software development:
  - ✓any error in this phase would affect all subsequent phases of development.
- Consists of two different activities:
  - ✓Requirements gathering and analysis
  - ✓Requirements specification

Summary

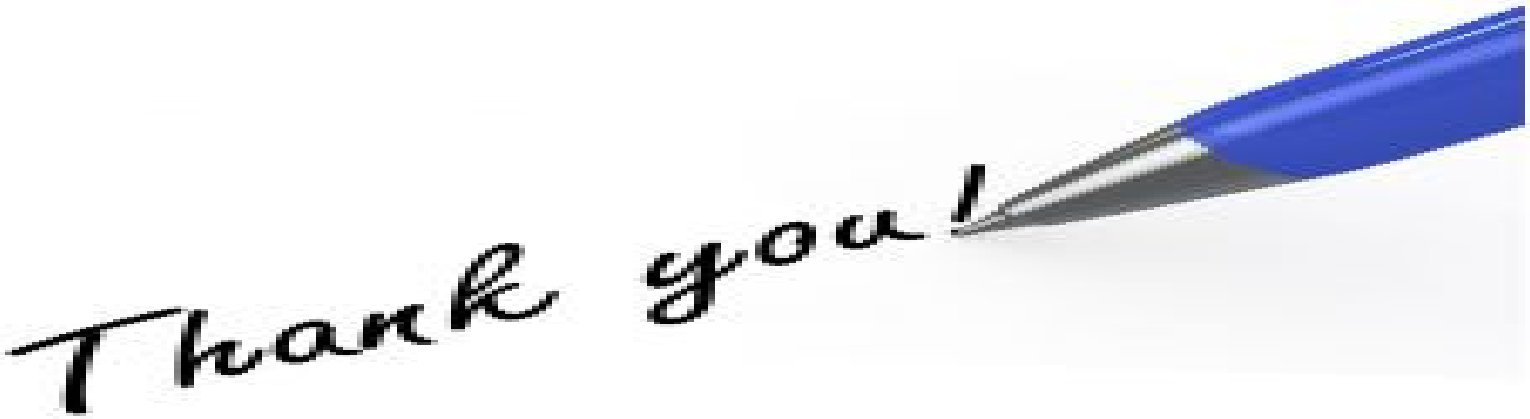
- The aims of requirements analysis:
  - ✓Gather all user requirements
  - ✓Clearly understand exact user requirements
  - ✓Remove inconsistencies and incompleteness.
- The goal of specification:
  - ✓ systematically organize requirements
  - ✓document the requirements in an SRS document.

Summary

- Main components of SRS document:
  - ✓functional requirements
  - ✓nonfunctional requirements
  - ✓constraints
- Techniques to express complex logic:
  - ✓Decision tree
  - ✓Decision table

Summary

- Formal requirements specifications have several advantages.
  - ✓But the major shortcoming is that these are hard to use.


Thank you!